MetaMan utility user guide

BASE24-eps®

trusted globally

ACI payment systems

Contents

About this document

This manual provides information that enables you to configure and manage BASE24-eps® metadata (e.g., the ASSIGNS file, CONFCSV and MDBCSV) using the MetaMan utility. The utility runs from a command prompt. MetaMan commands, which can be entered on the command line or in a command file, enable you to set maximum file size, path, and table information; add components, assigns, parameters, and derived types; and generate data files and scripts.

Audience

The ***BASE24-eps MetaMan Utility User Guide*** is intended for system managers responsible for configuring and maintaining BASE24-eps metadata. This guide also is for programmers who develop customizations to BASE24-eps.

Syntax notation

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
lowercase italics	Identify variables that you must provide.
Brackets [ ]	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar (|).
Vertical bar |	Separates alternatives in a list of items.
Ellipsis …	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Notation	Meaning
Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbol.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

What's new

Here is what's new in the March 2012 edition of the ***MetaMan utility user guide.***

Section	Description
1	Updates the graphic to include the DB2 LUW database.
2	Updates the type and base-type variable descriptions for the Component Definition File, Parameters Definition File, Derived Types Definition File formats to include the DB2 LUW database.
3	Updates the Setpath Data command and Setpath OLTP command to include the DB2 LUW database. Updates the database and data_src_type variables for the following commands to include the DB2 LUW database: • Setpath Jrnl • Settbl • Generate CONFCSV • Generate MDBCSV • Generate Assigns • Generate Script (WO 080609-28)

1: Introduction

The ACI Metadata Manager (MetaMan) utility is a metadata management tool that defines disk file structures and their associated data elements. The standard application configuration files it creates are provided in the initial delivery of BASE24-eps. However, the MetaMan utility can be used by developers to create portable data dictionary representations of data elements within the file systems of BASE24-eps. It also can be used to define files needed to tailor the BASE24-eps system to customer's unique requirements (e.g., Context files for interchange interfaces and journal definitions).

The MetaMan utility is a BASE24-eps Data Access Layer (DAL) tool that is used with all system platforms. System administrators use the MetaMan utility to create configuration information, such as data elements and tables, used by BASE24-eps. The MetaMan utility uses text-based files that are easily customized and stored.

The organization of BASE24-eps metadata allows you to update and customize your BASE24-eps system without the assistance of ACI personnel.

Custom component definition files are maintained independently of product component definition files and are included with minimal impact to the product stream. Keeping these files independent of each other makes it easier to add custom software modifications (CSMs) to your system and also retains the integrity of the product files. It is recommended that customers store MetaMan configuration files in their Source Code Management system.

The MetaMan utility also enables you to generate an assign configuration file on demand. You can then modify this file for physical file locations based on file organization scheme and used as input for subsequent runs of the utility to preserve your assign configuration.

The MetaMan utility recognizes CSMs, and they are carried forward as you upgrade your BASE24-eps system.

This user guide assumes that the BASE24-eps product has already been installed at your site. The installation procedure asks installers for file locations and other information that is used by the MetaMan utility.

MetaMan utility

You can use the MetaMan utility to create the configuration files that are used by the infrastructure for data access. These files connect the physical data to the logical data. The utility's conversational interface enables you to use (or *include*) system metadata to generate configuration files that are easy to maintain as your BASE24-eps system changes.

The MetaMan utility can create the following configuration files:

ASSIGNS—Assigns map logical table names as they are known to BASE24-eps to physical locations. The ASSIGNS file, along with all of your customizations, can be carried forward as you enhance your BASE24-eps system.

C—C language headers using the Generate C command. These headers are used by host programmers to create copybooks.

COBOL— COBOL language headers using the Generate COBOL command. The COBOL language headers are used by host programmers to create COBOL copybooks.

CONFCSV—The Metadata Configuration Comma Separated Values file (CONFCSV) contains the assigns used in the BASE24-eps system. Information includes the assign name, table name, table version, and physical file location. The BASE24-eps foundation layer uses this file to locate physical files and set certain processing parameters. The Unix platform calls this file the config.csv. For convenience, this file is referred to as CONFCSV throughout this guide.

MDBCSV—The Meta Database Comma Separated Values file (MDBCSV) contains the database tables created by the MetaMan utility. The metadata is accessed during processing from external memory tables. The table definition includes the table name, table version, and the record layout (all elements and keys). The BASE24-eps foundation layer uses this file to access data sources. The Unix platform calls the file the mdb.csv. For convenience, this file is referred to as MDBCSV throughout this guide.

METADH—The header file used to compile the BASE24-eps application. The MetaMan utility generates one directory that contains the header files for all tables. These files contain all elements defined, their lengths, and tables. These files are used by host programmers. You can generate this directory for headers associated with CSMs.

Scripts—Scripts that contains database creation commands also knowns as the physical file creation script are used to build files.

The MetaMan utility enables you to process customized configuration files. When you create a customized file (as opposed to modifying an existing file) the configurations are not overwritten by patches, service packs, and upgrades.

Utility overview

The MetaMan utility reads text files that contain metadata and generates the files that are used to run and maintain the BASE24-eps product.

ACI delivers BASE24-eps with a command file that directs the MetaMan utility to include the component table definitions for the components you have licensed. The command file contains commands that generate the output for application configuration, assigns, metadata headers, C or COBOL output, and database file creation scripts. As you add enhancements and CSMs to your system, you can add corresponding commands to the command file.

The following illustration shows the relationships among the input files, command files, and output files.

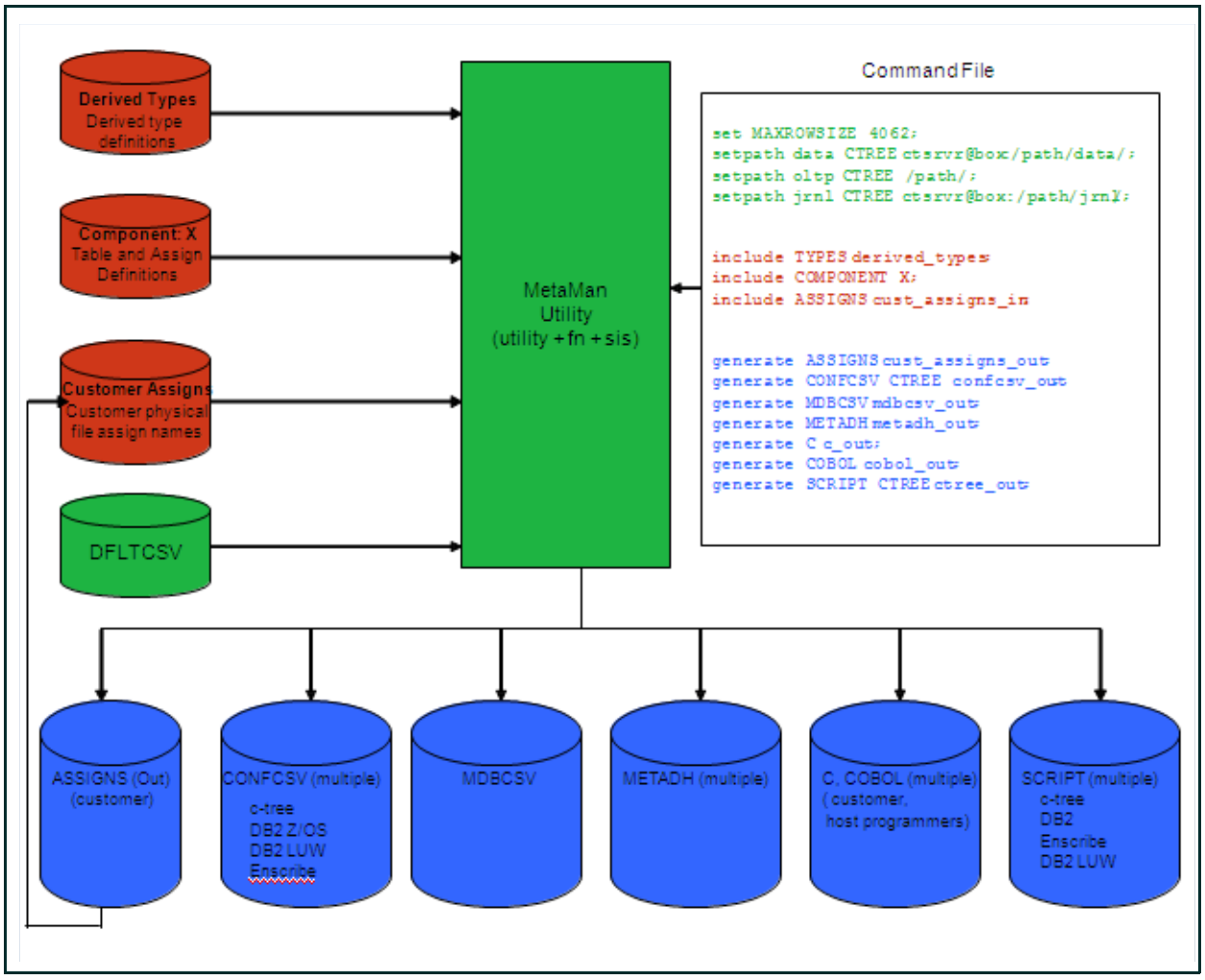

Figure 1: MetaMan utility overview

You also can use the conversational interface to enter individual commands instead of using the command file.

Input files

Data input files contain the metadata necessary to generate system output that is used by the BASE24-eps system. The input files include the following:

- Derived types (TYPES file) that keep data elements, such as account numbers, timestamps, and institution IDs, consistent throughout the system

- Table (ML and MQ files) and assign definitions (ASSIGNS file) for each licensed component
- Default CONFCSV and MDBCSV. The default CONFCSV contains the assigns to which the BASE24-eps system writes (i.e., anything that is generated). The default MDBCSV contains the “stream” that is used to write system output. On the Unix and IBM System z platforms, the defaults are in the config.csv and mdb.csv files which are stored in the metadata directory. On the HP NonStop platform the file is called CONFCSV.

ACI delivers the BASE24-eps product with the derived types, component files, and the default CONFCSV and MDBCSV (CONFCSV for HP NonStop platform, config.csv and mdb.csv for IBM System z and Unix platforms). You generate the ASSIGNS file with the first run of the MetaMan utility.

Note: ACI strongly recommends not changing any of the above files. If you want to add information (e.g., a CSM, additional journal files, or file partitions), ACI recommends using a custom component file.

Refer to the Input file formats section for samples of input file formats.

Naming conventions

Component files start with the letters ML (e.g., MLFND for the foundation component file). To easily identify customized files and to ensure that these files are sorted correctly by the esfix utility, ACI recommends that you use the MQ prefix when you create CSM files (e.g., use the name MQJRNL for customizations to the MLJRNL file). Note that the MQ prefix is for identification and sorting purposes and is not related to IBM Websphere MQ.

Command files

The command file is delivered with ACI code and contains the commands used to set physical file locations and generate data for all of the components that are part of the installation. This is a text file, and you can modify a copy with commands specific to your BASE24-eps system. You can continue to add components to the command file to ensure that changes you make are carried forward with each version of BASE24-eps. Refer to section 3 for more information about these commands.

Output

Running the MetaMan utility enables you to generate output that is used for running and maintaining your BASE24-eps system. MetaMan output includes the following:

- ASSIGNS file. The assign file contains the physical file location. This is an editable file which can be used as input for future runs of the utility.
- CONFCSV.
- MDBCSV.

- METADH.
- C and COBOL definitions (i.e., the physical layout of the table).
- Scripts (e.g., the physical file creation script). After the scripts have been generated, you must load them using the RUNCTBLD command (for Unix systems) or FUP (for HP NonStop systems).

BASE24-eps reads the CONFCSV and MDBCSV from external memory. Refer to the ***BASE24-eps Environment Management User Guide*** for information about how to build external memory tables (EMTs) for these files.

Product delivery

ACI delivers the BASE24-eps product with all input files for the components you have licensed, with the exception of the ASSIGNS file which is generated when the utility is run as part of the installation program. During the installation process, the installation program adds default file locations for your site to the input files. After the locations are added to the input files, the installation program generates the output files, including the ASSIGNS file. Note that all assigns of a certain type in the ASSIGNS file point to a single location. For example, all data files would be in one location, the OLTP files in a second location, and the Journal files in a third location. If you want to store files in different location, you must modify a copy of the ASSIGNS file with the new physical file locations and run the MetaMan utility again with the new ASSIGNS file as input (i.e., in an Include command). Refer to section 3 for more information about including and generating the ASSIGNS file.

Sample command file

ACI delivers the commands you need to get started in command files. The command file or files delivered with your ACI code contain the Set, Setpath, Settbl, Include, and Generate commands for the modules you have licensed. The commands for a BASE24-eps system running on the HP NonStop platform are in a single file called metacmd. The commands for the IBM System z or Unix platforms are delivered in three different files:

- The obey_cmd file contains platform- and database-specific information.
- The obey_comp file contains the list of component files (identified by the ML prefix)
- The obey_gen file contains the commands to generate the metadata files

The following is a sample of a command file for a BASE24-eps system running on an HP NonStop platform that has licensed an ATM Device Handler.

```
SET APPLICATION ES;
SETPATH DATA ENSCRIBE $D004.ES111DATA.;
SETPATH OLTP ENSCRIBE $D004.ES111DATA.;
SETPATH JRNL ENSCRIBE $D004.ES111JRNL.;
INCLUDE COMPONENT $D004.ES111META.MLDTYP;
INCLUDE COMPONENT $D004.ES111META.MLFND;
```

```
INCLUDE COMPONENT $D004.ES111META.MLCTX;
INCLUDE COMPONENT $D004.ES111META.MLINTF;
INCLUDE COMPONENT $D004.ES111META.MLPRFX;
INCLUDE COMPONENT $D004.ES111META.MLACQ;
INCLUDE COMPONENT $D004.ES111META.MLBUS;
INCLUDE COMPONENT $D004.ES111META.MLCRD;
INCLUDE COMPONENT $D004.ES111META.MLATMDH;
.
.
.
generate ASSIGNS ENSCRIBE ASSIGNS;
generate CONFCSV ENSCRIBE CONFCSV_ENSCRIBE;
generate MDBCSV MDBCSV;
generate SCRIPT ENSCRIBE SCRIPT_ENSCRIBE;
```

Note that in this example, the component file names are fully qualified. However, if you start the MetaMan utility from the same directory as the configuration files, the file names do not have to be fully qualified. If you run the MetaMan utility from the a different directory, you must fully qualify the file names.

Note: ACI strongly recommends keeping the command file or files used at installation intact. You can use a copy of a command file as a model for any CSMs you add to your system.

Customizing data paths

The Setpath commands specify the paths used when the ASSIGNS file is generated. The commands are customized for your site (i.e., they contain the paths used by your site). Although the files are grouped by file type, you can still customize the data paths further. For example, if you want to have the Card file in a location different from the rest of your files, you can modify the ASSIGNS file that is generated to include a new location for the Card file.

Default CONFCSV

Anything that is generated must already have an assign in the default CONFCSV. For users on the IBM System z platform or the Unix platform, the config.csv is in the ENV_VARS file. For the HP NonStop platform you must specify the location of the default CONFCSV when you start the MetaMan utility. Refer to section 3 for information about loading the ENV_VARS file and starting the MetaMan utility.

Maintaining metadata

Each time configuration information is added or changed, you must run the MetaMan utility to update the BASE24-eps metadata.

You should use the following input files each time you run the MetaMan utility:

- ASSIGNS file
- Component files
- Derived types files
- Default CSV (CONFCSV and MDBCSV for HP Non Stop or config.csv and mdb.csv for IBM System z and Unix)

Note that ACI does not supply an ASSIGNS table. The ASSIGNS table is created on the first run of the MetaMan utility from the Component files. To ensure that the customizations you have made to your system's metadata get carried forward, you must include the most current version of the ASSIGNS file (i.e., your edited or customized file) on subsequent runs of the MetaMan utility. If you do not include the edited ASSIGNS file, the MetaMan utility overwrites your customizations.

The following table summarizes when it is necessary to run the MetaMan utility and the output files that need to be re-created at various production milestones.

Production milestone	Run required when...	Output files
Installation (running MetaMan for the first time)	Always required	CONFCSV MDBCSV Scripts ASSIGNS file
Updating ASSIGNS file	If you are changing locations or making other edits to the ASSIGNS file	CONFCSV Scripts
BASE24-eps service pack	There are new Component files	CONFCSV MDBCSV Scripts ASSIGNS (if you want to specify the location of the new component tables)
New version of BASE24-eps	There are new Component (i.e., ML) files	CONFCSV MDBCSV Scripts ASSIGNS file
Adding BASE24-eps enhancements	There are new Component (i.e., ML) files	CONFCSV MDBCSV Scripts ASSIGNS file

Production milestone	Run required when...	Output files
CSM	There are customizations to an existing table or you are adding a new table	ASSIGNS file (if you are changing file locations) CONFCSV MDBCSV METADH (if you need to generate headers) Scripts

ASSIGNS file usage

The ASSIGNS file is used for both input and output. The MetaMan utility creates the ASSIGNS file the first time you run the utility. Since the MetaMan utility creates the ASSIGNS file as text, you can easily make customizations to the file
(e.g., change file locations) and include it in subsequent runs of the utility to ensure that configuration information is carried forward in the CONFCSV, MDBCSV, and other output files. Do not make changes directly to the CONFCSV or MDBCSV files. Because these files are generated, you would lose your modifications the next time you ran the MetaMan utility.

2: Input File Formats

ACI delivers BASE24-eps with input files customized for your system. While ACI recommends that you keep the original files intact, you can use them as models for further customization of your system.

Assign definition file format

A custom ASSIGNS file that contains the file assigns in the format below is generated the first time you run the MetaMan utility. That ASSIGNS file can be used as input on subsequent runs of the MetaMan utility to assign the proper file locations when building the CONFCSV.

The ASSIGNS file contains one assign for each data source used for the components you have licensed. Use this file as input when you are changing the location of a file. ACI recommends making a copy of the ASSIGNS file (e.g., CUSTASGN) before making any modification to it as a protection in case you need to go back to the original one (this preserves your physical file location configuration). You can then modify the assign in the new file. On future runs of the MetaMan utility, you can specify the CUSTASGN file as input (in the Include command) and output (in the Generate command). This preserves your physical file location configuration. Refer to section 3 for information about formatting Include and Generate commands.

To add assigns, ACI recommends using the Component file, described later in this section.

The format of the ASSIGNS definition file is shown below, followed by descriptions of its variables.

```
CREATE ASSIGN assign_name
    (
    TABLE_NAM table_name,
    TABLE_VERSION table_version,
    [DATA_SRC_TYP data_src_type,]
    PHYSICAL_FILE_NAM physical_file_name,
    [PRELOAD_FROM_ASSIGN_NAME preload_table_name,]
    [CONFIG_STRING -MT_SIZE=config_string]
    );
```

where:

assign_name—The name of the assign (e.g., ATM_MAC).

table_name—The name of the table.

table_version—A string representing the table version value (e.g., 1.0). The ASSIGNS file that is generated when you install BASE24-eps and any ASSIGNS file you receive from ACI are always whole number versions
(e.g., 1.0, 2.0, etc.). When you alter a table for a custom software modification (CSM), you can change the version to a decimal value (e.g., 1.1 for the first alteration to product version 1.0, 1.2 for the second alteration to product version 1.0). The MetaMan utility uses the record with the highest version number. If ACI delivers a new version of the ASSIGNS file, you must create a new version if you want to retain your alterations.

data_src_type—The data source type value. Valid values are as follows:

HASHDS	=	Generic hash table data source
DB2CLIqrep	=	DB2LUW database Q replication (IBM InfoSphere Replication Server)
DB2zOSqrep	=	DB2 database Q replication (IBM zOS InfoSphere Replication Server)
WOP	=	Write only pipe
RADIXDS	=	Generic radix table data source
WOF	=	Write only file
MQDS	=	Websphere MQ data source
SIAUTODS	=	Auto commit data source

physical_file_name—The physical file name value. This field contains the full path to the physical file based on your file organization scheme. Locations in the ASSIGNS file override default file locations that may have been included in the Setpath commands when generating the configuration.

preload_table_name—The preload table name value. Used for OLTP files.

config_string—The maximum number of bytes to allocate to build the OLTP memory tables.

Component definition file format

The Component definition file contains table definitions and assign definitions. Each component of the BASE24-eps product has a Component Definition file. The component definition file contains one entry for each table used by the component.

To create Component definition files for customized components (for example, adding a new table or adding elements to an existing table), ACI recommends placing customizations in a new component file that contains the new information and name it with an MQ prefix. Do not alter the product Component Definition files. You can then add the new Component Definition file to the command file after the original entry. For example, if you created a new Journal file, MQJRNL, you would include the file after the command for the original MLJRNL file.

The table format of the Component definition file is shown below, followed by descriptions of its variables. Use the format for the ASSIGNS file described earlier in this section to add new file assigns.

```
//
//  Comments describing the purpose of this table.
//
CREATE TABLE table_name
    (          Start of table definition
    TABLE_VERSION table_version,
    SIS_TBL_SHORT_NAM short_name,
    SIS_TBL_AUDIT_IND y/n,
    [SIS_TBL_MAX_ROW_LGTH nnnn,]
    SIS_TBL_LEGACY_IND y/n,
    //
       // <elementn description>
    //
    <element namen> <type> [DELTA] [DEFAULT <default value>],
    PRIMARY_KEY key_name Pkey_abbrev [UNIQUE] (element1,
         elementn)
    ALTERNATE_KEY <key name> <Akey abbrev1>, [UNIQUE] (<element1>,
                                                        <elementn>)
    ...
    ALTERNATE_KEY <key name> <Akey abbrev>, [UNIQUE] (<element1>,
                                                       <elementn>)
    );           End of table definition
```

where

table_name—The name of the table (e.g., ATM_MAC).

table_version—A string representing the table version value (e.g., 1.0). Any component file you receive from ACI is always a whole number version (e.g., 1.0, 2.0, etc.). When you create a table for a CSM (i.e., a file with an MQ prefix), you can use a whole number for the version (e.g., 1.0). If you want to add elements to an existing table, you must use an ALTER TABLE command. Refer to the topic, “Alter Table Command Format” for more information about using ALTER TABLE commands to customize component files.

short_name—The short name or physical file name of the component table.

y/n—The letter (y for yes, n for no) that indicates whether a table audit is used. If you are using the autocommit data source type, this value must be set to n.

nnnn—The table maximum record size.

y/n—The letter (y for yes, n for no) that indicates whether the table is a legacy table (i.e., a table that already exists in the system).

elementn_description—The description of the element in use in the table.

elementn_name —The name of the element followed optionally by the delta and default value.

type—The type of data element. Valid values are as follows:

bt_binaryf = Fixed length string, binary
bt_binaryl = Two-byte data length followed by variable length string, binary
bt_binaryv = Variable length string, binary
bt_char = Character
bt_flag = Boolean
bt_float32 = Four-byte decimal point
bt_float64 = Eight-byte decimal point
bt_int16s = Two-byte integer
bt_int32s = Four-byte integer
bt_int64s = Eight-byte integer
bt_stringf = Fixed length string
bt_stringv = Variable length string

The SIS rule for metadata is that there can only be one stringv or binaryv field in a table and it must be the last field in the table so that DAL can find it. If there are more than one, then the binaryl data type must be used.

DELTA—An optional literal value that indicates whether the element can be replicated to another site as a whole final value or an intermediate (delta) value. If this value is present, the element can be replicated as intermediate value.

DEFAULT*default_value*—The default value of the element.

key_name—The name of the primary key.

Pkey_abbrev—The primary key abbreviation.

element1, elementn—The list of elements that make up the primary key. Values are separated by commas.

UNIQUE—A literal that indicates that the value of the keys cannot be replicated. If this value is not present, you do not have a unique key (i.e., your system allows duplicate records).

ALTERNATE_KEY—The keyword indicating an alternate key.

key_name—The name of the alternate key.

Akey_abbrev—The alternate key abbreviation.

The following is a sample of a component definition file format, in this case, for the Stream table.

```
//
// The Stream table contains one record per line to be written to
// a write only data source.  The line is composed of a single
// variable length string.
//
CREATE TABLE Stream
```

```
    (
    TABLE_VERSION 1.0,
    SIS_TBL_SHORT_NAM strmd,
    SIS_TBL_AUDIT_IND N,
    SIS_TBL_LEGACY_IND Y,
    SIS_TBL_MAX_ROW_LGTH 4062,
    //
    // The stream element is a variable length string to be
    // written to the write only data source.
    //
    stream_element bt_stringv(4062)
    );
```

Alter table command format

To add an element to an existing table, you can enter an ALTER TABLE command into a new file. Putting extensions into a new file (named with an MQ prefix) ensures that future versions of the file do not overwrite your extensions. The format of the ALTER TABLE command in a file is shown below.

```
ALTER TABLE table-name
    (         Start of alter definition
    TABLE_VERSION <n.0>,
    //
    //  element-namex-description
    //
    ADD element-namex type [DELTA] [DEFAULT default-value],
    ...
    //
    //  element-namey-description
    //
    ADD element-namey type [DELTA] [DEFAULT default-value]
    ALTERNATE_KEY key-name Akey-abbrev1, [UNIQUE] (element1,
                                                   elementn)
    //
    // ALTER_PRIMARY_KEY and ALTER_ALTERNATE_KEY allow
    // to csm existing key, adding elements to it
    //
    ALTER_ALTERNATE_KEY key_name Pkey_abbrev [UNIQUE]
     (New elementx,New elementy)
    ALTER_ALTERNATE_KEY <key name> <Akey abbrev1>, [UNIQUE]
     (New elementx,New elementy)
    );        End of alter definition
```

The *table-name* variable must match the name of the table that you are altering. Enter a whole number for the highest table version in the product a the time the ALTER TABLE command is created (e.g., 1.0). The MetaMan utility combines the version number in the ALTER TABLE command with the version number of the table. For example, if you are on version 1.0 of a table, and you create an ALTER TABLE command for the table, the table number in the MQ* file is 1.0. When you run the MetaMan utility, the MDBCSV has an entry for version 1.1 and 1.0.

The MetaMan utility automatically rolls to new versions of altered files. Continuing with the example above, when you get a new version of the table from ACI (version 2.0) and use the MetaMan utility to generate a new MDBCSV, the MetaMan automatically creates version 2.1 of the MDBCSV along with versions 2.0, 1.1, and 1.0.

New ALTER TABLE commands must use the current product version of the table based on the BASE24-eps version being used. New ALTER TABLE commands must be added to the end of a series of ALTER TABLE commands for your CSMs. If you want to add another element to the table in the above example, set the version number of the ALTER TABLE command to 2.0. The MetaMan utility automatically generates a 2.2 version of the table in the MDBCSV (but not a version 1.2).

After running an ALTER TABLE command on a table that is in production, the ALTER TABLE command cannot be deleted or changed. Because there is a dependency on the order of ALTER TABLE commands to generate the *.n* versions of each table, and the associated table version that is assigned by the MetaMan utility, the ALTER TABLE commands must be executed in the same order each time.

ACI recommends naming new elements with a prefix such as CSM or a customer acronym to avoid future conflicts with product element names.

The following is an example of an ALTER TABLE command.

```
ALTER TABLE Prefix
    (
    TABLE_VERSION 7.0,

    //
    // An identifier used to show the PIN verification profile to which
    // this record belongs on the PIN verification data source (e.g., IBM
    // DES, Diebold, Identikey, Visa PVV).  If this field is not blank,
    // the Authorization processes will use the value of this field,
    // among others,to read the appropriate PIN verification record.  If
    // this field contains blanks, then the Authorization processes will
    // not perform PIN verification.  Valid values are any combination of
    // alphanumeric characters and leading and trailing spaces.
    //
    ADD pinv_keya_prfl prfl_def 1.0,

    //
    // PIN Digits Verify KEYA Profile. An identifier for the profile to
    // which this record applies. This field must contain a value that
    // matches a PIN Verify Profile defined in the Prefix.
    //
    ADD pinv_dgt_keya_prfl prfl_def 1.0,

    // The point at which the Bad PIN Tries usage is reset.  The process
    // compares the value of the Bad PIN Tries usage to the value in the
    // MAX-PIN-TRY limit.  Valid values are:  0 = The value of the Bad
    // PIN Tries usage is reset with each usage period. (Default)   1 =
    // The value of the Bad PIN Tries usage is reset when a good PIN is
    // entered providing the MAX-PIN-TRY limit has not been reached. It
    // is also reset with each usage period.  2 = The value of the Bad
```

```
    // PIN  Tries usage is reset regardless of the value of the MAX-PIN-
    // TRY limit value. It is also reset with each usage period.  3 = The
    // value of the Bad PIN Tries usage is reset when a good PIN is
    // entered providing the MAX-PIN-TRY limit has not been reached. It
    // is not reset with each usage period. 4 = The value of Bad PIN
    //Tries  usage is reset regardless of the value of the MAX-PIN-TRY
    // limit value. It is not reset with each usage period.
    //
    ADD pin_try_reset_opt bt_stringf(1),

    //
    // The number of times that the processes will allow a cardholder to
    // attempt to enter their PIN correctly before declining the request
    // and taking the action specified by the value in the BAD-PIN-DISP
    // field. Valid values are 0-999.
    //
    ADD max_pin_try bt_int16s,

    //
    // The action which the processes will take when a cardholder has
    // exceeded the allowable attempts to enter the correct PIN.  This
    // field is only used if the PV-KEYA-GRP field is not equal to
    // blanks.  Valid values: 0 = Return the card, 1 = Retain the card
    //
    ADD bad_pin_disp bt_stringf(1),

    //
    // PIN Verification Value Length
    //
    ADD tvv_lgth bt_int16s

    );
```

Parameters definition file format

The Parameters definition file is included as part of a run of the MetaMan utility. The file contains one entry for each configuration parameter.

The format of the Parameters definition file is shown below, followed by descriptions of its variables.

```
CREATE PARAM param_name
    (         ! Start of parameter definition
    PARAM_VERSION version_n.n,
    PARAM_TYPE type,
    [PARAM_VALUE value]
    [DERIVED_TYP_NAM <value>]
    );        ! End of parameter definition
```

where

param_name—The name of the parameter.

version_n.n—A string representing the parameter version value (e.g., 1.0). Any PARAM file you receive from ACI is always a whole number version (e.g., 1.0, 2.0, etc.).

type—The type of parameter. Valid values are as follows:

bt_binaryf = Fixed length string, binary
bt_binaryl = Two-byte data length followed by variable length string, binary
bt_binaryv = Variable length string, binary
bt_char = Character
bt_flag = Boolean
bt_float32 = Four-byte decimal point
bt_float64 = Eight-byte decimal point
bt_int16s = Two-byte integer
bt_int32s = Four-byte integer
bt_int64s = Eight-byte integer
bt_stringf = Fixed length string
bt_stringv = Variable length string

The SIS rule for Metadata is that there can only be one stringv or binaryv field in a table and it must be the last field in the table so that DAL can find it. If there are more than one, then the binaryl data type must be used.

PARAM_VALUE *value*—A literal value indicating the parameter value, followed by the value in double quotation marks (e.g., PARAM_VALUE “1000”). A param value is optional.

DERIVED_TYP_NAME *value*—A literal value indicating a derived type name value, followed by the value in double quotation marks. A derived type name value is optional.

The following is a sample of the Parameters Definition file format:

```
CREATE PARAM prfxod_MAX_CHAIN_IDX
    (
    PARAM_VERSION 1.0,
    PARAM_TYPE bt_stringf(28),
    PARAM_VALUE 28
    );
```

Derived types definition file format

The following file format is used to define the derived types used by your BASE24-eps system. The Derived types file contains one entry for each derived type. The format is followed by descriptions of its variables.

```
//
//  Derived type description.
//
CREATE TYPE type-name
    (         Start of type definition
    TYPE_VERSION type_version,
    BASE_TYPE base-type,
    [DEFAULT default-value]
    );        End of type definition
```

type-name—A literal value indicating a derived type, followed by the name of the derived type

type_version—A string representing the type version value (e.g., 1.0). Any TYPE file you receive from ACI is always a whole number version (e.g., 1.0, 2.0, etc.).

base-type—A literal that identifies the base type, followed by the derived type's base type. Valid values are as follows:

bt_binaryf = Fixed length string, binary
bt_binaryl = Two-byte data length followed by variable length string, binary
bt_binaryv = Variable length string, binary
bt_char = Character
bt_flag = Boolean
bt_float32 = Four-byte decimal point
bt_float64 = Eight-byte decimal point
bt_int16s = Two-byte integer
bt_int32s = Four-byte integer
bt_int64s = Eight-byte integer
bt_stringf = Fixed length string
bt_stringv = Variable length string

The SIS rule for Metadata is that there can only be one stringv or binaryv field in a table and it must be the last field in the table so that DAL can find it. If there are more than one, then the binaryl data type must be used.

DEFAULT *default-value*—A literal indicating a default value, followed by the derived type's default value. A default value is optional.

The following is a sample of the Derived Types file definition.

```
//
// Channel Identifier definition.
//
CREATE TYPE chan_id
    (
    TYPE_VERSION 1.0,
    BASE_TYPE bt_stringf(16)
    );
```

3: Commands

This section describes the commands you can use to configure and maintain system metadata using the MetaMan utility. It also describes input file formats.

You can enter MetaMan commands at the command prompt or from a command file containing multiple commands.

MetaMan commands enable you to add data to the configuration files used by the system; set name, table size, and path information; and generate metadata.

Adding data to the BASE24-eps system

Use Include commands to add data to the BASE24-eps system.

Note that all file names specified in the Include command must be fully qualified.

Include component command

The Include Component command adds a single component to the configuration. Additional component names are used for stream and environment components. When adding Include Component commands for a CSM (i.e., those that include a file with an MQ prefix) to a command file, enter them after the Include command for the original entry.

Syntax

INCLUDE COMPONENT *file-path file-name*;

file-path — The physical file location of the new component. This is case sensitive, depending on platform.

file-name — The name of the component file. This is case sensitive, depending on platform.

Examples

include component /PATH/MLPRFX;

include component $vol.subvol.MLPRFX;

Include Assigns command

The Include Assigns command enables you to specify the ASSIGNS file used to generate configuration data (e.g., the CONFCSV and MDBCSV). You can use an Include command to customize the ASSIGNS file after the initial run of the MetaMan utility. The initial run of the MetaMan utility produces an Assign file with default physical file locations. You can modify the default names using an Include command for each file location. The customized ASSIGNS input file is used as input to generate the new CONFCSV (assign to physical file location relationships). ASSIGNS included as input take precedence over the default assigns built using data from a Setpath command.

ACI strongly recommends using a copy of the ASSIGNS file instead or modifying the original file.

Syntax

INCLUDE ASSIGNS *assigns-input*;

assigns-input—The physical file location of the ASSIGNS file. This is case sensitive, depending on platform.

Examples

include ASSIGNS /PATH/CUSTASGN_IN;

include ASSIGNS $vol.subvol.CUSTASGN;

Include Types command

The Include Types command adds derived database types to the BASE24-eps configuration files. If your BASE24-eps system uses derived database types, you must include derived database type definitions every time you run the MetaMan utility.

Syntax

INCLUDE TYPES *derived-types-assign*;

derived-types-assign—The physical file location of the derived types file. This is case sensitive, depending on platform.

Example

include TYPES /PATH/DERIVED_TYPES;

include TYPES $vol.subvol.DERTYPES;

Include Params command

The Include Params command adds configuration parameters.

Syntax

INCLUDE PARAMS *param-assign*;

param-assign—The physical file location of the params file. This is case sensitive, depending on platform.

Examples

include PARAMS /PATH/PARAM-IN;

include PARAMS $vol.subvol.PARAMS;

Setting name, table, and path information

A Set command sets the application name and the maximum table size information. The Setpath command specifies a default location of the data, journal, and OLTP files. The Settbl command changes the table name, data source type, path information and/or config information for all assigns using a specific table name.

Set Application command

The Set Application command specifies the application name that is placed at the beginning of the MDBCSV file output.

Syntax

SET APPLICATION *application-name*;

application-name —The name of the application. This is case sensitive, depending on platform.

Example

set APPLICATION TSS;

Note: If you use this command before the generation of the MDBCSV is requested, a default application name of ES should be used.

Set Maxrowsize command

The Set Maxrowsize command sets the maximum row size for each table. The value set by this command is used as the row size if one is not supplied as part of the table creation definition (i.e., the SIS_TBL_MAX_ROW_LGTH ATTRIBUTE in the create table statement). If the maximum row size is not set, the utility uses a default value of 4062 as the maximum row size. Refer to vendor documentation for more information about the maximum row size for each platform.

Syntax

SET MAXROWSIZE *nnnn*;

nnnn—The number of positions in the row.

Example

set MAXROWSIZE 6000;

Setpath Data command

The Setpath Data command sets the default physical location (i.e., path) used by the MetaMan utility to expand the physical file names for each database type. The default data path is used when building the disk file output, physical file name when the assign or file names does not exist in the assign file.

Syntax

SETPATH DATA *database database-path*;

database—The type of database. Valid values are as follows:

CTREE = c-tree database
DB2LUW = DB2 LUW database
DB2ZOS = IBM System z DB2 database
ENSCRIBE = HP NonStop Enscribe database

database-path—The physical file location of the database. This is case sensitive, depending on platform.

Examples

setpath DATA CTREE CTSRVR@BOX:/PATH/DATA/;

setpath DATA ENSCRIBE $VOL.SUBVOL.;

Setpath OLTP command

The Setpath OLTP command sets the default physical location path that is used by the MetaMan utility to expand the physical file names for each database type. The default OLTP path is used when building the OLTP file output, physical file name when the assign or file name does not exist in the assign file.

Syntax

SETPATH OLTP *database database-path;*

database—The type of database. Valid values are as follows:

```
CTREE     = c-tree database
DB2LUW    = DB2 LUW database
DB2ZOS    = IBM System z DB2 database
ENSCRIBE  = HP NonStop Enscribe database
```

database-path—The physical file location of the database. This is case sensitive, depending on platform.

Examples

setpath oltp DB2ZOS ./OLTP/;

setpath oltp CTREE /PATH/;

setpath oltp ENSCRIBE $VOL.SUBOLTP.;

Setpath JRNL command

The Setpath JRNL command sets the default physical location path used by the MetaMan utility to expand the physical file names for each database type. The default journal path is used when building journal disk file output, physical file name when the assign or file name does not exist in the assign file.

Syntax

SETPATH JRNL *database database-path*;

database—The type of database. Valid values are as follows:

```
CTREE      = c-tree database
DB2LUW     = DB2 LUW database
DB2ZOS     = IBM System z DB2 database
ENSCRIBE   = HP NonStop Enscribe database
```

database-path—The physical file location of the database. This is case sensitive, depending on platform.

Example

setpath jrnl CTREE CTSRVR@BOX:/PATH/JRNL/;

setpath jrnl ENSCRIBE $VOL.SUBJRNL.;

Settbl command

The Settbl command changes the table name, data source type, path information, and configuration information for all assigns using a specific table name.

Syntax

SETTBL *database existing_table_name*, *new-table-name* [,*data_src_type*, *physical-file-name-prefix*, *config-string*]

database—The type of database. Valid values are as follows:

```
CTREE      = c-tree database
DB2LUW     = DB2 LUW database
DB2ZOS     = IBM System z DB2 database
db2zosqrep = DB2 database Q replication (InfoSphere Replication Server)
ENSCRIBE   = HP NonStop Enscribe database
```

existing_table_name—The name of the table as it exists in the metadata.

new-table-name—The new name of the table. Note that this is a required variable that must be used in the command regardless of whether there is a change in table name.

data_src_type—The data source type value. Valid values are as follows:

HASHDS	=	Generic hash table data source
DB2CLIqrep	=	DB2 database Q replication for a DB2LUW database (IBM InfoSphere Replication Server).
DB2zOSqrep	=	DB2 database Q replication (InfoSphere Replication Server)
WOP	=	Write only pipe
ENSCRIBE	=	HP NonStop Record Manager
RADIXDS	=	Generic radix table data source
WOF	=	Write only file
MQDS	=	Websphere MQ data source
SIAUTODS	=	Auto commit data source

physical-file-name-prefix—The prefix of the physical file name defined by the Setpath command.

config_string—The string to be configured in the new table.

Examples

SETTBL db2zos Context, Context, MQDS, B482., "-TRACE=OFF -OLDREC=300"

SETTBL db2zos Journal, Journal_Key_Seq2

Generating configuration files

The Generate commands enable you to produce specific types of metadata output.

The Generate commands are described below. A sample output file follows each command description.

Generate CONFCSV command

The Generate CONFCSV command creates the CONFCSV. CONFCSV data includes the table assigns configuration, and assign physical file name.

Syntax

GENERATE CONFCSV *database assign*;

database—The type of database. Valid values are as follows:

CTREE = c-tree database

DB2LUW = DB2 LUW database
DB2ZOS = IBM database
ENSCRIBE = HP NonStop Enscribe database

assign—The name of the CONFCSV assign. This is case sensitive, depending on platform.

Examples

generate CONFCSV DB2ZOS CONFCSV_OUT;

generate CONFCSV CTREE CONFCSV_OUT;

generate CONFCSV ENSCRIBE CONFCSV_OUT;

generate CONFCSV DB2LUW CONFCSV_OUT;

Output

The following is a sample of the output from the Generate CONFCSV command.

```
"Config"

"AppName","ConfigName","DateTime","UserName"
"ES","ES_ENSCRIBE","04/14/2011 10:38:14","admn"

"Assigns"

"AssignName","TableName","TableVersion","PhysicalName","DataSourceType","Confi
gString","PreloadFromAssignName"
"ABI_INTERFACE","ABI_Interface","1.0","$vol.subvol.abifd","Enscribe","",""
"ABI_INTERFACE_OLTP","ABI_Interface_OLTP","1.0","$vol.suboltp.
abifod","HASHDS","-MT_SIZE=500000","ABI_INTERFACE"
"ABI_KEY","ABI_Key","1.0","$vol.subvol.abikd","Enscribe","",""
"ABI_KEY_OLTP","ABI_Key_OLTP","1.0","$vol.suboltp.abikod","HASHDS","-MT_
SIZE=500000","ABI_KEY"
"ACCEL_INTERFACE","ACCEL_Interface","1.0","$vol.subvol.acifd","Enscribe","",""
"ACCEL_INTERFACE_OLTP","ACCEL_Interface_OLTP","1.0","$vol.suboltp.
acifod","HASHDS","-MT_SIZE=500000","ACCEL_INTERFACE"
"ACQ_ISS_RELATION","Acquirer_Issuer_Relation","1.0","$vol.subvol.
aird","Enscribe","",""
"ACQ_RTE_PROFILE","Acquirer_Route_Profile","1.0","$vol.subvol.
artpd","Enscribe","",""
"ACQ_TXN_ALLOWED_SHMT","Acquirer_Txn_Allowed","1.0","$vol.subvol.
acqtxd","SHMT","",""
"ACQ_TXN_ALLOWED_XCF","Acquirer_Txn_Allowed","1.0","$vol.subvol.
acqtxd","XCF","",""
"ACQUIRER_ISSUER_RELATION_OLTP","Acquirer_Issuer_Relation_OLTP","1.0","$vol.
suboltp.airod","HASHDS","-MT_SIZE=500000","ACQ_ISS_RELATION"
"ACQUIRER_TXN_ALLOWED_OLTP","Acquirer_Txn_Allowed_OLTP","1.0","$vol.suboltp.
aqtxod","HASHDS","-MT_SIZE=500000","ACQ_TXN_ALLOWED"
.
.
.
```

```
"Parameters"

"ElementName","ElementVersion","ParamValue","BaseTypeName","Size","MaxDimensio
n","VariableLengthFlag","DerivedTypeName"
"CICS_JES_CLASS_ID","1.0","E","bt_stringf",1,70,False,""
"CICS_JES_JCL_ID","1.0","","bt_stringf",1,70,False,""
"CICS_JES_NODE_ID","1.0","N1","bt_stringf",1,70,False,""
"CICS_TRACE_DATA","1.0","N","bt_stringf",1,70,False,""
"CICS_TRACE_FNCT","1.0","N","bt_stringf",1,70,False,""
"CICS_TRACE_PROC","1.0","N","bt_stringf",1,70,False,""
"cpfd_DUPS","1.0","N","bt_stringf",1,1,False,"req"
"cpfd_KEY_TYPE","1.0","N","bt_stringf",1,1,False,"req"
"cpfd_MAX_CHAIN_IDX","1.0","28","bt_stringf",1,28,False,"pan"
"ipfd_DUPS","1.0","N","bt_stringf",1,1,False,"req"
"ipfd_KEY_TYPE","1.0","N","bt_stringf",1,1,False,"req"
"ipfxod_DUPS","1.0","N","bt_stringf",1,1,False,""
"ipfxod_KEY_TYPE","1.0","N","bt_stringf",1,1,False,""
"ipfxod_MAX_CHAIN_IDX","1.0","28","bt_stringf",1,28,False,""
"prfxod_DUPS","1.0","N","bt_stringf",1,1,False,""
"prfxod_KEY_TYPE","1.0","N","bt_stringf",1,1,False,""
"prfxod_MAX_CHAIN_IDX","1.0","28","bt_stringf",1,28,False,""
"scpfd_DUPS","1.0","Y","bt_stringf",1,1,False,""
"SIMDS_DEST_FILENAME","1.0","$vol.subvol.mdsdest","bt_stringf",1,32,False,""
"SIMDS_DEST_WARMBOOT_DELTA","1.0","0","bt_stringv",1,5,True,""
"strmd_CLASS_DATA","1.0","A","bt_stringf",1,32,False,""
"strmd_NODE_DATA","1.0","N1","bt_stringf",1,32,False,""
"strmd_PAGE_LENGTH","1.0","132","bt_stringf",1,32,False,""
"strmd_USERID_DATA","1.0","CICSUSER","bt_stringf",1,32,False,""
```

Generate MDBCSV command

You can create the MDBCSV using the Generate MDBCSV command. MDBCSV data includes the table assigns configuration, and assign physical file name.

Syntax

GENERATE MDBCSV *assign*;

assign—The name of the MDBCSV assign. This is case sensitive, depending on platform.

Examples

generate MDBCSV MDBCSV_OUT;

Output

The following is sample output from the Generate MDBCSV command.

```
"App"

"GenAppId","DateTime","UserName","AppName","DataSourceType","NumTables","Singl
eTableFlag","Description"
1,"04/14/2011 10:38:15","admn","ES","",655,False,"Enterprise Services. To be
created by importing all other applications only by the Architecture Team"

"Tables"

"GenAppId","GenTableId","TableName","TableVersion","SmallName","MaxPhysicalRec
ordSize","EndOfFixedFields","NumLogicals","NumKeys","PriKeyName","PriKeyTinyNa
me","PriKeyOffset","PriKeyLength","SISTblVer","LegacyFileFlag","AuditFlag"
1,1,"ABI_Interface","1.0","abifd",110,110,25,1,"prikey","PK",4,16,1,False,True
1,2,"ABI_Interface_OLTP","1.
0","abifod",98,98,23,1,"prikey","PK",0,16,1,True,False
1,3,"ABI_Key","1.0","abikd",36,36,4,1,"prikey","PK",4,16,1,False,True
1,4,"ABI_Key_OLTP","1.0","abikod",24,24,2,1,"prikey","PK",0,16,1,True,False
1,5,"ACCEL_Interface","1.0","acifd",45,45,5,1,"prikey","PK",4,16,1,False,True
1,6,"ACCEL_Interface_OLTP","1.
0","acifod",33,33,3,1,"prikey","PK",0,16,1,True,False
1,7,"Acct_Status_Extrn_to_Intrn","1.
0","aseid",80,80,6,1,"prikey","PR",4,18,1,False,True
1,8,"Acct_Status_Intrn_to_Extrn","1.
0","asied",80,80,6,1,"prikey","PR",4,18,1,False,True
1,9,"Acquirer_Issuer_Relation","1.
0","aird",62,62,6,1,"prikey","PK",4,34,1,False,True
1,10,"Acquirer_Issuer_Relation_OLTP","1.
0","airod",50,50,4,1,"prikey","PK",0,34,1,True,False
1,11,"Acquirer_Route_Profile","1.
0","artpd",60,60,4,1,"prikey","PK",4,16,1,False,True
1,12,"Acquirer_Txn_Allowed","1.
0","acqtxd",39,39,8,1,"prikey","PK",4,23,1,False,True
1,13,"Acquirer_Txn_Allowed_OLTP","1.
0","aqtxod",27,27,6,1,"prikey","PK",0,23,1,True,False
.
.
.
"Elements"

"GenAppId","GenTableId","LogicalOrdinal","NumCopies","LogicalCopyNum","PhysOrd
inal","ElmtName","BaseTypeName","BaseTypeSize","ElmtDim","Offset","Length","Di
mFlag","VarLenFlag","PadFlag","OffsetFlag","OverHeadBytes","DfltVal","DeltaFla
g"
1,1,1,1,1,1,"sis_tbl_ver","bt_
int32s",4,1,0,4,False,False,False,False,0,"1",False
1,1,2,1,1,2,"intf_nam","bt_
stringf",1,16,4,16,True,False,False,False,0,"",False
1,1,3,1,1,3,"fm_ts","bt_int64s",8,1,20,8,False,False,False,False,0,"",False
1,1,4,1,1,4,"atm_ssb","bt_flag",1,1,28,1,False,False,False,False,0,"",False
1,1,5,1,1,5,"hndlr_repeat_msg","bt_
flag",1,1,29,1,False,False,False,False,0,"",False
1,1,6,1,1,6,"abi_setl_dat","bt_
flag",1,1,30,1,False,False,False,False,0,"",False
1,1,7,1,1,7,"abi_e100_gc","bt_
flag",1,1,31,1,False,False,False,False,0,"",False
```

```
1,1,8,1,1,8,"pos_send_1220","bt_
flag",1,1,32,1,False,False,False,False,0,"",False
1,1,9,1,1,9,"pos_send_moto_1220","bt_
flag",1,1,33,1,False,False,False,False,0,"",False
1,1,10,1,1,10,"dbi_field_44","bt_
flag",1,1,34,1,False,False,False,False,0,"",False
1,1,11,1,1,11,"abi_cc_rrn","bt_
flag",1,1,35,1,False,False,False,False,0,"",False
1,1,12,1,1,12,"abi_orig_crncy","bt_
flag",1,1,36,1,False,False,False,False,0,"",False
1,1,13,1,1,13,"abi_abl_srvc","bt_
flag",1,1,37,1,False,False,False,False,0,"",False
.
.
.
"KeyElements"
```

"GenAppId","GenTableId","KeyId","KeyOrdinal","KeyName","TinyName","KeyUniqueFlag","KeyPrimaryFlag","LogicalOrdinal","ElementName","LogicalCopyNum","PhysicalOrdinal"

```
1,1,1,1,"prikey","PK",True,True,2,"intf_nam",1,2
1,2,1,1,"prikey","PK",True,True,1,"intf_nam",1,1
1,3,1,1,"prikey","PK",True,True,2,"intf_nam",1,2
1,4,1,1,"prikey","PK",True,True,1,"intf_nam",1,1
1,5,1,1,"prikey","PK",True,True,2,"intf_nam",1,2
1,6,1,1,"prikey","PK",True,True,1,"intf_nam",1,1
1,7,1,1,"prikey","PR",True,True,2,"acct_stat_prfl",1,2
1,7,1,2,"prikey","PR",True,True,3,"acct_stat_extrn",1,3
1,8,1,1,"prikey","PR",True,True,2,"acct_stat_prfl",1,2
1,8,1,2,"prikey","PR",True,True,3,"acct_stat",1,3
1,9,1,1,"prikey","PK",True,True,2,"iss_rte_prfl",1,2
1,9,1,2,"prikey","PK",True,True,3,"acq_rte_prfl",1,3
1,9,1,3,"prikey","PK",True,True,4,"txn_cde",1,4
1,10,1,1,"prikey","PK",True,True,1,"iss_rte_prfl",1,1
1,10,1,2,"prikey","PK",True,True,2,"acq_rte_prfl",1,2
1,10,1,3,"prikey","PK",True,True,3,"txn_cde",1,3
1,11,1,1,"prikey","PK",True,True,2,"acq_rte_prfl",1,2
1,12,1,1,"prikey","PK",True,True,2,"acq_txn_prfl",1,2
1,12,1,2,"prikey","PK",True,True,3,"msg_cat",1,3
1,12,1,3,"prikey","PK",True,True,4,"proc_cde",1,4
1,13,1,1,"prikey","PK",True,True,1,"acq_txn_prfl",1,1
1,13,1,2,"prikey","PK",True,True,2,"msg_cat",1,2
1,13,1,3,"prikey","PK",True,True,3,"proc_cde",1,3
```

Generate METADH command

You can generate the METADH language-specific output headers for each table using the Generate METADH command. The headers are used for compiling the BASE24-eps application.

Syntax

GENERATE METADH *metadh_assign*;

metadh_assign—The name of the METADH assign. This is case sensitive, depending on platform.

Example

generate METADH METADH_ASSIGN;

Output

The following is sample output for the Generate METADH command.

```
/* ************************************************************************
   This File Name              : ES.h
   Metadata header for App     : ES
   Generation Date             : 04/14/2011 10:38:26
   Generated by user           : admn
   Number of tables generated  : 655
   ************************************************************************ */
#ifndef STDLIB_H
#include <stdlib.h>
#ifndef STDLIB_H
#define STDLIB_H
#endif
#endif

/* ------------------------------------------------------------------------
Table: ABI_Interface (abifd)

*/
#ifndef ABIFD_H
#define ABIFD_H

#define tbl_abifd_d                     "ABI_Interface"

#define      abifd_sis_tbl_ver_d        \
            "sis_tbl_ver"
#define      abifd_sis_tbl_ver_ord_d    \
            1
const size_t abifd_sis_tbl_ver_lgth     = 4  ;//long

#define      abifd_intf_nam_d           \
            "intf_nam"
#define      abifd_intf_nam_ord_d       \
            2
.
.
```

```
.
*/
#define key_abifd_prikey                    "prikey"

#endif /* #ifndef ABIFD_H */
```

Generate C command

You can generate C language headers using the Generate C command. These headers are used by host programmers to create copybooks.

Syntax

GENERATE C *C_ASSIGN*;

C_ASSIGN—The name of the C assign. This is case sensitive, depending on platform.

Example

generate C C_OUT;

Output

The following is sample output from the Generate C command.

```
#ifndef ES_HPP
#define ES_HPP

/*
 * Table       : ABI_Interface (abifd)
 * TableVersion: 1.0
 * SISTblVer   : 1
 */
typedef struct ABI_Interface_def_
    {
    unsigned char sis_tbl_ver[4];                                   /* long */
    unsigned char intf_nam[16];                                     /* char[16]
*/
    unsigned char fm_ts[8];                                         /* __int64 */
    unsigned char atm_ssb;                                          /* bool */
    unsigned char hndlr_repeat_msg;                                 /* bool */
    unsigned char abi_setl_dat;                                     /* bool */
    unsigned char abi_e100_gc;                                      /* bool */
    unsigned char pos_send_1220;                                    /* bool */
    unsigned char pos_send_moto_1220;                               /* bool */
    unsigned char dbi_field_44;                                     /* bool */
    unsigned char abi_cc_rrn;                                       /* bool */
    unsigned char abi_orig_crncy;                                   /* bool */
```

```
    unsigned char abi_abl_srvc;                                 /* bool */
    unsigned char pos_fl2_type;                                 /* bool */
    unsigned char pos_dest_circuit_inst_id[11];                 /* char[11]
*/
    unsigned char pos_tran_orig_inst_id[11];                    /* char[11]
*/
    unsigned char atm_rcv_inst_id[11];                          /* char[11]
*/
    unsigned char pos_rcv_inst_id[11];                          /* char[11]
*/
    unsigned char atm_abi_frwd_inst_id[11];                     /* char[11]
*/
    unsigned char abi_multi_key[1];                             /* char[1] */
    unsigned char pos_dflt_rtlr[1];                             /* char[1] */
    unsigned char pos_rcncl_hour[2];                            /* short */
    unsigned char pos_rcncl_min[2];                             /* short */
    unsigned char pos_restart_1520_tim[2];                      /* short */
    unsigned char abi_bin_prestitempo[8];                       /* char[8] */
    } ABI_Interface_def;

/*
 * Table       : ABI_Interface_OLTP (abifod)
 */
typedef struct ABI_Interface_OLTP_def_
    {
    unsigned char intf_nam[16];                                 /* char[16]
*/
    unsigned char atm_ssb;                                      /* bool */
    unsigned char hndlr_repeat_msg;                             /* bool */
    unsigned char abi_setl_dat;                                 /* bool */
    unsigned char abi_e100_gc;                                  /* bool */
    unsigned char pos_send_1220;                                /* bool */
    unsigned char pos_send_moto_1220;                           /* bool */
    unsigned char dbi_field_44;                                 /* bool */
    unsigned char abi_cc_rrn;                                   /* bool */
    unsigned char abi_orig_crncy;                               /* bool */
    unsigned char abi_abl_srvc;                                 /* bool */
    unsigned char pos_fl2_type;                                 /* bool */
    unsigned char pos_dest_circuit_inst_id[11];                 /* char[11]
*/
    unsigned char pos_tran_orig_inst_id[11];                    /* char[11]
*/
    unsigned char atm_rcv_inst_id[11];                          /* char[11]
*/
    unsigned char pos_rcv_inst_id[11];                          /* char[11]
*/
    unsigned char atm_abi_frwd_inst_id[11];                     /* char[11]
*/
    unsigned char abi_multi_key[1];                             /* char[1] */
    unsigned char pos_dflt_rtlr[1];                             /* char[1] */
    unsigned char pos_rcncl_hour[2];                            /* short */
    unsigned char pos_rcncl_min[2];                             /* short */
    unsigned char pos_restart_1520_tim[2];                      /* short */
    unsigned char abi_bin_prestitempo[8];                       /* char[8] */
    } ABI_Interface_OLTP_def;
```

Generate COBOL command

You can generate COBOL language headers using the Generate COBOL command. The COBOL language headers are used by host programmers to create COBOL copybooks.

Syntax

GENERATE COBOL *COBOL_assign*;

COBOL_assign—The name of the COBOL assign. This is case sensitive, depending on platform.

Example

generate COBOL COBOL_OUT;

Output

The following is sample output from the Generate COBOL command.

```
 * COBOL FILE DEFINITIONS
      * APP NAME         : ES
      * GENERATION DATE  : 04/14/2011 10:38:25
      * GENERATED BY USER: ADMN
      * NUMBER OF TABLES : 655

       01 ABI-INTERFACE.
         10 SIS-TBL-VER                    PIC S9(9) COMP.
         10 INTF-NAM                       PIC X(16).
         10 FM-TS                          PIC S9(18) COMP.
         10 ATM-SSB                        PIC X.
         10 HNDLR-REPEAT-MSG               PIC X.
         10 ABI-SETL-DAT                   PIC X.
         10 ABI-E100-GC                    PIC X.
         10 POS-SEND-1220                  PIC X.
         10 POS-SEND-MOTO-1220             PIC X.
         10 DBI-FIELD-44                   PIC X.
         10 ABI-CC-RRN                     PIC X.
         10 ABI-ORIG-CRNCY                 PIC X.
         10 ABI-ABL-SRVC                   PIC X.
         10 POS-FL2-TYPE                   PIC X.
         10 POS-DEST-CIRCUIT-INST-ID       PIC X(11).
         10 POS-TRAN-ORIG-INST-ID          PIC X(11).
         10 ATM-RCV-INST-ID                PIC X(11).
         10 POS-RCV-INST-ID                PIC X(11).
         10 ATM-ABI-FRWD-INST-ID           PIC X(11).
         10 ABI-MULTI-KEY                  PIC X(1).
         10 POS-DFLT-RTLR                  PIC X(1).
         10 POS-RCNCL-HOUR                 PIC S9(4) COMP.
         10 POS-RCNCL-MIN                  PIC S9(4) COMP.
         10 POS-RESTART-1520-TIM           PIC S9(4) COMP.
         10 ABI-BIN-PRESTITEMPO            PIC X(8).
```

Generate Assigns command

Use the Generate Assigns command to create an ASSIGNS output file that can be used for subsequent runs of the MetaMan utility. The ASSIGNS file can be modified based on your organization scheme. After you modify the file, you can move it to the cust_assign__in location to be used for subsequent runs of the utility.

Syntax

GENERATE ASSIGNS *database assigns_out*;

database—The type of database. Valid values are as follows:

```
CTREE      = c-tree database
DB2LUW     = DB2 LUW database
DB2ZOS     = IBM System z DB2 database
ENSCRIBE   = HP NonStop Enscribe database
```

assigns_out—The name of the assigns_out file. This is case sensitive, depending on platform.

Examples

```
generate ASSIGNS CTREE CUST_ASSIGNS_OUT;
generate ASSIGNS DB2ZOS CUST_ASSIGNS_OUT;
generate ASSIGNS DB2LUW CUST_ASSIGNS_OUT;
generate ASSIGNS ENSCRIBE CUST_ASSIGNS_OUT;
```

Output

The following is sample output from the Generate Assigns command.

```
//
*******************************************************************************
//  Create Table script for App : ES
//  Generation Date             : 04/14/2011 10:38:23
//  Generated by user           : admn
//  Number of tables generated  : 655
//
*******************************************************************************
//  The following table assignments are created by this script:
//  TableName TableVersion              ShortName  Type            Alternate
Keys
//      AssignName                          PhysicalName
// ------------------------------------------------------------------------
-
//  ABI_Interface 1.0                   abifd      Key Sequence    0
//      ABI_INTERFACE                       abifd
//  ABI_Interface_OLTP 1.0              abifod     Key Sequence    0
//      ABI_INTERFACE_OLTP                  abifod
//  ABI_Key 1.0                         abikd      Key Sequence    0
//      ABI_KEY                             abikd
//  ABI_Key_OLTP 1.0                    abikod     Key Sequence    0
//      ABI_KEY_OLTP                        abikod
//  ACCEL_Interface 1.0                 acifd      Key Sequence    0
//      ACCEL_INTERFACE                     acifd
//  ACCEL_Interface_OLTP 1.0            acifod     Key Sequence    0
//      ACCEL_INTERFACE_OLTP                acifod
//  Acquirer_Issuer_Relation 1.0        aird       Key Sequence    0
//      ACQ_ISS_RELATION                    aird
//  Acquirer_Route_Profile 1.0          artpd      Key Sequence    0
```

```
//      ACQ_RTE_PROFILE                     artpd
.
.
.
REATE ASSIGN ABI_INTERFACE
    (
    TABLE_NAME ABI_Interface,
    TABLE_VERSION 1.0,
    PHYSICAL_FILE_NAM $vol.subvol.abifd
    );

CREATE ASSIGN ABI_INTERFACE_OLTP
    (
    TABLE_NAME ABI_Interface_OLTP,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE HASHDS,
    PHYSICAL_FILE_NAM $vol.suboltp.abifod,
    PRELOAD_FROM_ASSIGN_NAME ABI_INTERFACE,
    CONFIG_STRING "-MT_SIZE=500000"
    );

CREATE ASSIGN ABI_KEY
    (
    TABLE_NAME ABI_Key,
    TABLE_VERSION 1.0,
    PHYSICAL_FILE_NAM $vol.subvol.abikd
    );

CREATE ASSIGN ABI_KEY_OLTP
    (
    TABLE_NAME ABI_Key_OLTP,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE HASHDS,
    PHYSICAL_FILE_NAM $vol.suboltp.abikod,
    PRELOAD_FROM_ASSIGN_NAME ABI_KEY,
    CONFIG_STRING "-MT_SIZE=500000"
    );
CREATE ASSIGN ACCEL_INTERFACE
    (
    TABLE_NAME ACCEL_Interface,
    TABLE_VERSION 1.0,
    PHYSICAL_FILE_NAM $vol.subvol.acifd
    );

CREATE ASSIGN ACCEL_INTERFACE_OLTP
    (
    TABLE_NAME ACCEL_Interface_OLTP,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE HASHDS,
    PHYSICAL_FILE_NAM $vol.suboltp.acifod,
    PRELOAD_FROM_ASSIGN_NAME ACCEL_INTERFACE,
   CONFIG_STRING "-MT_SIZE=500000"
    );
```

```
CREATE ASSIGN ACQ_ISS_RELATION
    (
    TABLE_NAME Acquirer_Issuer_Relation,
    TABLE_VERSION 1.0,
    PHYSICAL_FILE_NAM $vol.subvol.aird
    );

CREATE ASSIGN ACQ_RTE_PROFILE
    (
    TABLE_NAME Acquirer_Route_Profile,
    TABLE_VERSION 1.0,
    PHYSICAL_FILE_NAM $vol.subvol.artpd
    );
```

Generate Script command

You can generate database creation scripts used to build files with the Generate Script command.

Syntax

GENERATE SCRIPT *database assign*;

database—The type of database. Valid values are as follows:

CTREE = c-tree database
DB2LUW = DB2 LUW database
DB2ZOS = IBM System z DB2 database
ENSCRIBE = HP NonStop Enscribe database

assign—The name of the script assign. This is case sensitive, depending on platform.

Examples

generate SCRIPT CTREE CTREE_OUT;

generate SCRIPT DB2ZOS DB2ZOS_OUT;

generate SCRIPT DB2UW DB2LUW_OUT;

generate SCRIPT ENSCRIBE ENSCRIBE_OUT;

Output

The following is sample output for a DB2 script.

```
-- ------------------------------------------------------------------------
-
-- *  App              : ES
-- *  Description      : Enterprise Services. To be created by importing all
-- *                     other applications only by the Architecture Team
-- *
-- *  Configuration    : ES_DB2zOS_TESTBED
-- *  Description      : Enterprise Services Configuration - DB2zOS - Testbed
-- *  Generation Date  : 04/14/2011 10:38:35
-- *  Generated by user: admn
-- *  Number of table  : 655
-- ------------------------------------------------------------------------
-
-- *  The following file assignments are generated by this script:
-- *  Assign Name                      Table Name
-- *    Short Name Physical Name
-- *  ABI_INTERFACE                    ABI_Interface
-- *    abifd      abifd
-- *  ABI_KEY                          ABI_Key
-- *    abikd      abikd
-- *  ACCEL_INTERFACE                  ACCEL_Interface
-- *    acifd      acifd
-- *  ACQ_ISS_RELATION                 Acquirer_Issuer_Relation
-- *    aird       aird
-- *  ACQ_RTE_PROFILE                  Acquirer_Route_Profile
-- *    artpd      artpd
-- *  ACT_CDE_EXT_INT                  Action_Code_Extrn_to_Intrn
-- *    aceid      aceid
-- *  ACT_CDE_INT_EXT                  Action_Code_Intrn_to_Extrn
-- *    acied      acied
-- *  ACTFUNC                          ActFunc
-- *    uactfn     uactfn
-- *  ACTSESS                          ActSess
-- *    uactse     uactse
-- *  ACTV_SCRIPT_STAT                 Active_Script_Statistics
-- *    actssd     actssd
.
.
.
-- CREATE DATABASE #DBNAMEEPS#
--    BUFFERPOOL BP5
--    INDEXBP BP2
--    STOGROUP #SGNAME#;

--
*************************************************************************
--  Table:       ABI_Interface (abifd)
--  File ABI_INTERFACE created on 04/14/2011 10:38:35
--
*************************************************************************
-- DROP TABLE #CREATOR#.abifd;
```

```
-- DROP TABLESPACE #DBNAMEEPS#.abifd;

CREATE TABLESPACE abifd
    IN #DBNAMEEPS#
    USING STOGROUP #SGNAME#
    PRIQTY 12
    SECQTY 12
    FREEPAGE 0
    PCTFREE 5
    SEGSIZE 32
    BUFFERPOOL BP5;

CREATE TABLE #CREATOR#.abifd
    (
    intf_nam              CHAR(16)
        NOT NULL,
    fm_ts                 CHAR(8)
        FOR BIT DATA
        NOT NULL DEFAULT X'0000000000000000',
    atm_ssb               CHAR(1)
        NOT NULL DEFAULT '0',
    hndlr_repeat_msg      CHAR(1)
        NOT NULL DEFAULT '0',
    abi_setl_dat          CHAR(1)
        NOT NULL DEFAULT '0',
    abi_e100_gc           CHAR(1)
        NOT NULL DEFAULT '0',
    pos_send_1220         CHAR(1)
        NOT NULL DEFAULT '0',
    pos_send_moto_1220    CHAR(1)
        NOT NULL DEFAULT '0',
    dbi_field_44          CHAR(1)
        NOT NULL DEFAULT '0',
    abi_cc_rrn            CHAR(1)
        NOT NULL DEFAULT '0',
    abi_orig_crncy        CHAR(1)
        NOT NULL DEFAULT '0',
    abi_abl_srvc          CHAR(1)
        NOT NULL DEFAULT '0',
    pos_fl2_type          CHAR(1)
        NOT NULL DEFAULT '0',
    pos_dest_circuit_inst_id  CHAR(11)
        NOT NULL DEFAULT '',
    pos_tran_orig_inst_id  CHAR(11)
        NOT NULL DEFAULT '',
    atm_rcv_inst_id       CHAR(11)
        NOT NULL DEFAULT '',
    pos_rcv_inst_id       CHAR(11)
        NOT NULL DEFAULT '',
    atm_abi_frwd_inst_id  CHAR(11)
        NOT NULL DEFAULT '',
    abi_multi_key         CHAR(1)
        NOT NULL DEFAULT '',
    pos_dflt_rtlr         CHAR(1)
        NOT NULL DEFAULT '',
```

```
    pos_rcncl_hour        SMALLINT
        NOT NULL DEFAULT 0,
    pos_rcncl_min         SMALLINT
        NOT NULL DEFAULT 0,
    pos_restart_1520_tim  SMALLINT
        NOT NULL DEFAULT 0,
    abi_bin_prestitempo   CHAR(8)
        NOT NULL DEFAULT '',
    PRIMARY KEY
        (
        intf_nam
        )
    ) IN #DBNAMEEPS#.abifd;

    CREATE UNIQUE INDEX #CREATOR#.abifd_PK ON #CREATOR#.abifd
        (
        intf_nam
        )
        CLUSTER
        USING STOGROUP #SGNAME#
        PRIQTY 12
        SECQTY 12
        FREEPAGE 0
        PCTFREE 5
        CLOSE NO;

    GRANT SELECT, INSERT, UPDATE, DELETE ON #CREATOR#.abifd
    TO #GRANT1#;

--
*********************************************************************
--  Table:       ABI_Interface_OLTP (abifod)
--  File ABI_INTERFACE_OLTP created on 04/14/2011 10:38:35
--
*********************************************************************
--
*********************************************************************
--  Table:       ABI_Key (abikd)
--  File ABI_KEY created on 04/14/2011 10:38:35
--
*********************************************************************
-- DROP TABLE #CREATOR#.abikd;
-- DROP TABLESPACE #DBNAMEEPS#.abikd;

CREATE TABLESPACE abikd
    IN #DBNAMEEPS#
    USING STOGROUP #SGNAME#
    PRIQTY 12
    SECQTY 12
    FREEPAGE 0
    PCTFREE 5
    SEGSIZE 32
    BUFFERPOOL BP5;
```

```
CREATE TABLE #CREATOR#.abikd
    (
    intf_nam              CHAR(16)
        NOT NULL,
    fm_ts                 CHAR(8)
        FOR BIT DATA
        NOT NULL DEFAULT X'0000000000000000',
    abi_mac_key           CHAR(8)
        NOT NULL DEFAULT '',
    PRIMARY KEY
        (
        intf_nam
        )
    ) IN #DBNAMEEPS#.abikd;

    CREATE UNIQUE INDEX #CREATOR#.abikd_PK ON #CREATOR#.abikd
        (
        intf_nam
        )
        CLUSTER
        USING STOGROUP #SGNAME#
        PRIQTY 12
        SECQTY 12
        FREEPAGE 0
        PCTFREE 5
        CLOSE NO;

    GRANT SELECT, INSERT, UPDATE, DELETE ON #CREATOR#.abikd
    TO #GRANT1#;
.
.
.
```

4: Procedures

This section describes procedures for using the MetaMan utility and customization procedures to add tables and assigns to the metadata configuration files.

Procedures

The MetaMan utility runs from a command interface from which you can execute the command file or issue separate commands. The following topics describe how to load the ENV_VARS file (IBM System z and Unix platforms only), start the MetaMan utility, execute commands, and view output. You can find detailed information about the individual commands later in this section.

Load the ENV_VARS file

If your BASE24-eps system runs on the IBM System z or Unix platform you must load the ENV_VARS file before logging on to the MetaMan utility. The ENV_VARS file contains system variables required by the MetaMan utility to run, including the CONFCSV.

To load the file, enter the following at the system-level command prompt:

```
. . /env_vars
```

Refer to the platform-specific ***Operations Guide*** for more information about the ENV_VARS file.

Start the MetaMan utility

Start the MetaMan utility by following the steps described below for your platform. If the Include commands in the command file are not fully qualified, or if you do not want to enter fully qualified file names at the command line, you must change the directory to the location of the metadata files.

IBM System z or Unix platform

Enter the following command at the system-level command prompt:

```
runmeta
```

HP NonStop platform

Enter the following at the TACL prompt:

```
metamn config-file-location
```

where *config-file-location* is the location of the CONFCSV configuration file.

Execute individual commands

If your database changes are few, you can use the interface to execute individual commands rather than using the command file.

To execute a command, enter the command text as described later in this section at the MetaMan prompt.

Get help

To display help for the MetaMan utility (a list of commands available for the utility), enter the help command as shown below:

```
help
```

To get help for a specific command, use the following command:

```
help command-name
```

where *command-name* is the name of a MetaMan utility command.

Fix a command

If you have made a mistake entering a command (such as a typographical error), you can enter a command to fix it (instead of entering the wholde command again). Enter the following command to fix an error on the command line:

```
FC [number | string]
```

where *number* is the prompt number of a previously entered command and *string* is some or all of the characters with which a previously entered command begins.

If you enter neither the *number* or *string* variables, the last command is redisplayed.

The command accepts the following editing characters:

- D (delete). The character above the D is deleted.
- I (insert). The text following the I is inserted to the left of the character immediately above the I.
- R (replace). The text following the R replaces the text in the command line starting with the character immediately above the R

If more than one change is made at one time, the text strings must be terminated with two slashes (//).

Obey the command file

If you have a command file, you can use an obey command to execute all of the commands in the file. Use the following command on all platforms:

```
obey command-filename;
```

where *command-filename* is the fully qualified name of the command file. Note that on the IBM System z and Unix platform, you must enter a series of commands. The fully qualified command files are as follows:

Platform	Fully qualified file name
IBM System z or Unix	$DB/metadata/obey_cmd $DB/metadata/obey_comp $DB/metadata/obey_assigns $DB/metadata/obey_gen
HP NonStop	$VOL.*prfx*META.METACMD, where *prfx* is the system prefix used at installation (e.g., ES111)

For the sequence of entering this command with viewable output, see the procedures for viewing output below.

View output

When you enter individual commands from the command line, the output is displayed on the screen.

If you are executing a command file, you can use the following steps to log output to an output file for viewing.

Steps

1. Enter the following at the MetaMan command prompt:

```
LOG TO output_filename;
```

where:

output_filename is the name of the file to which the output data is to be written. Verify that an assign for this file exists in the CONFCSV.

2. Obey the command file.
3. Enter the following to stop logging:

```
LOG STOP;
```

4. View output from the file specified in step 1.

View history of commands entered

To see a list of the last 10 commands entered in a session, enter the following command:

```
HISTORY
```

To see a list of a specified number of commands entered in a session enter the following command:

```
HISTORY count
```

where *count* is the number of previous commands to display.

Delay a command

To delay execution of a MetaMan command for a specified amount of time (for example, on platforms such as the IBM where the execution of a large obey file can degrade the performance of the machine), enter the following command:

DELAY [*duration*]

where *duration* is the number of seconds to delay the command. If the *duration* variable is not specified, the default value of one second is used.

Exit the MetaMan utility

After you have run all of the commands, you can exit the application by entering EXIT at the prompt.

Move IBM System z and Unix metadata files to the configuration directory

On the IBM System z and Unix platforms, the MetaMan utility stores the output in temporary files in the $DB/b24oltp directory. You must move the following files to the $CONFIG directory:

- config.csv
- mdb.csv
- es.txt
- assigns

This step is not necessary for the HP NonStop platform, as the MetaMan utility stores them in the location from which they are used.

Build the CONFCSV and MDBCSV external memory tables

BASE24-eps uses external memory tables to access the CONFCSV and MDBCSV files during processing on the HP NonStop and Unix platforms. CONFEMT is the external memory table for CONFCSV information. MDBEMT is the external memory table for MDBCSV information. By accessing metadata from memory rather than from disk, performance is improved and new versions of data sources can be implemented in BASE24-eps processes on a rolling basis (one at a time) without incurring a complete outage of the entire BASE24-eps system.

Whenever you change data in the CONFCSV or MDBCSV, you must rebuild the associated external memory table using the Metadata EMT Build process. Refer to the ***BASE24-eps Environment management user guide*** for more information about the Metadata EMT Build process.

Customization procedures

To add tables and assigns to the metadata configuration files you can create a new component file that contains the new tables, elements and assigns and use the MetaMan utility to generate new ASSIGNS, CONFCSV, and MDBCSV files. Do not edit the CONFCSV and MDBCSV. If you make edits directly to these files, changes are overwritten by the esfix or esbldjnl programs.

Note: If you are not adding elements to a table or creating new table definitions (i.e., you are using the MetaMan utility to customize locations or configure options), ACI recommends using a custom ASSIGNS file instead of a custom component file.

Steps

1. Create a file that will hold the customizations required using the same format as the MLxxxx files, found in the directory ($DB/metadata for IBM System z and Unix platforms or *prfx*META for HP NonStop platforms), as a template. This file can contain assigns and table definitions. Save this file as MQxxxx, where xxxx is the name of the component (for example, MQJRNL for a customized version of the MLJRNL file). The MQ prefix differentiates this file from the product files and also indicates to the esfix program that this is a custom file to include when rebuilding the metadata during a fix.

   For example:

```
//
*************************************************************************
//  Create Table script for App : ES
//  Generation Date             : 06/10/2011 11:19:42
//  Generated by user           : admn
//  Number of tables generated  : 3, and 1 mod
//
*************************************************************************
//  The following table assignments are created by this script:
//  TableName TableVersion              ShortName  Type            Alternate
Keys
//      AssignName                          PhysicalName
// ------------------------------------------------------------------------
-
//  Limits_OLTP 1.0                     lmtod      Key Sequence    0
//      LIMITS_OLTP                         lmtod
//  Stream 1.0                          strmd      Entry Sequence  0
//      QUERYA                              rpta
//  Stream 1.0                          strmd      Entry Sequence  0
//      QUERYB                              rptb
//  Stream 1.0                          strmd      Entry Sequence  0
//      QUERYO                              rpto

CREATE ASSIGN LIMITS_OLTP
    (
    TABLE_NAME Limits_OLTP,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE HASHDS,
    PHYSICAL_FILE_NAM /db01/qatest/QA16/db/b24oltp/lmtod,
    PRELOAD_FROM_ASSIGN_NAME LIMITS,
    CONFIG_STRING "-MT_SIZE=1150000"
    );

CREATE ASSIGN QUERYA
    (
    TABLE_NAME Stream,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE WOF,
    PHYSICAL_FILE_NAM /db01/qatest/QA16/db/b24data/rpta
    );
```

```
CREATE ASSIGN QUERYB
    (
    TABLE_NAME Stream,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE WOF,
    PHYSICAL_FILE_NAM /db01/qatest/QA16/db/b24data/rptb
    );

CREATE ASSIGN QUERYO
    (
    TABLE_NAME Stream,
    TABLE_VERSION 1.0,
    DATA_SOURCE_TYPE WOF,
    PHYSICAL_FILE_NAM /db01/qatest/QA16/db/b24data/rpto
    );
```

2. Make a copy of the command file.
   - For IBM System z and Unix platforms, the command file is the obey_comp file in the $DB/metadata directory. Update it by adding an *Include* line for the new component (MQXXXX, in the sample below) at the end of the list of *Include* statements:

     include component MLVDPS;

     include component MLVISA;

     include component MLVSIP;

     include component MQXXXX;
   - For the HP NonStop platform the command file is the METACMD file in the *prfx*META subvolume. Update it adding an *Include* line for the new component (MQXXXX, in the sample below) at the end of the list of *Include* statements.

     INCLUDE COMPONENT $TST2.QATBMETA.MLVDPS;

     INCLUDE COMPONENT $TST2.QATBMETA.MLVISA;

     INCLUDE COMPONENT $TST2.QATBMETA.MLVSIP;

     INCLUDE COMPONENT $TST2.QATBCUST.MQXXXX;

     generate ASSIGNS ENSCRIBE ASSIGNS;

     generate CONFCSV ENSCRIBE CONFCSV_ENSCRIBE;

     generate MDBCSV MDBCSV;

     generate SCRIPT ENSCRIBE SCRIPT_ENSCRIBE;
3. Follow the procedures for starting the MetaMan utility and obeying the new command file or files as described earlier in this section. This deletes the current versions of the CONFCSV and MDBCSV files, and generates new copies that contain the customizations.
   - For the IBM System z and Unix platforms, the new files are generated to wherever the CONFCSV and MDBCSV locations are defined in the $DB/metadata/config.csv file. Normally, this is $DB/b24oltp.
   - For the HP NonStop platform, new CONFCSV and MDBCSV files are created on the configuration subvolume *prfx*cnfg, where *prfx* is the system prefix used at installation (e.g., ES111).
4. For the IBM System z and Unix platforms, move the config.csv, mdb.csv, es.txt, and assigns files to the $CONFIG directory

5. For the HP NonStop and Unix platforms, rebuild the CONFCSV and MDBCSV EMT tables as described in the ***BASE24-eps Environment Management User Guide.*** This step is not used on the IBM System z platform.

File maintenance example

The following example shows how to use the MetaMan Utility to create and maintain a customized assigns file with multiple Journal files. In this scenario, the customer adds three new journal files after installation and then adds two more at a later date. The name of the component file for the Journal component is MLJRNL. Note that these procedures do not include entering the assign names into the journal profiles. After these steps are completed, refer to the ***BASE24-eps Journal user guide*** for the steps for configuring journal profiles.

Create Journal files after initial installation

1. Create a component file containing the new Journal assigns, and give it a unique name (e.g., MQJRNL). The component file contains one entry for each of the new journal files.
2. Make a copy of the command file for reference purposes, and add an Include command for the new MQJRNL file after the Include command for the MLJRNL file.
3. Obey the command file using the MetaMan utility. The utility creates a new assigns file, CONFCSV, and MDBCSV.

Add more Journal files at a later date

1. Make a copy of the MQJRNL file for back-up purposes.
2. Edit the copy of the MQJRNL file to add assigns for the new journal files.
3. Verify that the command file contains an Include command for the MQJRNL file and your customized assigns file.
4. Obey the command file using the MetaMan utility. The utility creates a new assigns file, CONFCSV, and MDBCSV.

Index

O

P

S

T

ACI Worldwide
Offices in principal cities throughout the world.
www.aciworldwide.com

Americas +1 402 390 7600
Asia Pacific +65 6334 4843
Europe, Middle East,
Africa +44 (0) 1923 816393

About ACI Worldwide

ACI Worldwide powers electronic payments for financial institutions, retailers and processors around the world with the broadest, most integrated suite of electronic payment software in the market. More than 75 billion times each year, ACI's solutions process consumer payments. On an average day, ACI software manages more than US$12 trillion in wholesale payments. And for more than 150 payments organisations worldwide, ACI software ensures people and businesses don't fall victim to financial crime. We are trusted globally based on our unrivaled understanding of payments and related processes. We have a definitive vision of how electronic payment systems will look in the future and we have the knowledge, scale and resources to deliver it. Since 1975, ACI has provided software solutions to the world's innovators. We welcome the opportunity to do the same for you.